

Тема лабораторной работы 4 : Создание ETL-пайплайна для анализа данных

ETL означает Extract, Transform, Load. Значит, пайплайн должен извлекать данные из источника, преобразовывать их и загружать в хранилище для анализа. Нужно определить этапы: выбор источников данных, их извлечение, преобразование (очистка, агрегация и т.д.), загрузка в целевое хранилище. Также важно выбрать инструменты: может быть Python с библиотеками Pandas, Apache Airflow.

Сначала нужно выбрать данные для работы. Возможно, примером будут CSV-файлы или база данных. Затем написать код для извлечения данных. Потом этап трансформации: обработка пропущенных значений, преобразование типов, создание новых признаков. После этого загрузка в хранилище, например, в PostgreSQL или в облачное хранилище данных, как Google BigQuery.

Нужно учесть возможные ошибки: обработка исключений при извлечении данных, логирование, тестирование каждого этапа. Также можно добавить визуализацию данных после загрузки для анализа.

Следует предложить структуру проекта: разделить на папки для скриптов ETL, конфигурационных файлов, тестов. Использовать контроль версий, например, GitHub. Возможно, автоматизировать пайплайн с помощью Airflow для регулярного выполнения.

Важно в **readme** объяснить, почему каждый этап важен: извлечение гарантирует доступность данных, трансформация повышает качество данных, загрузка обеспечивает доступность для анализа. Возможно, привести пример кода на Python с использованием Pandas для ETL.

Также стоит упомянуть инструменты мониторинга и логирования, чтобы отслеживать выполнение пайплайна и вовремя обнаруживать ошибки. Возможно, интеграция с уведомлениями при сбоях.

Лабораторная работа: Создание ETL-пайплайна для анализа данных

Цель работы

Освоить принципы проектирования ETL-пайплайна (Extract, Transform, Load) для подготовки данных к анализу. Научиться извлекать данные из источника, преобразовывать их и загружать в целевую систему.

Задачи

Extract (Извлечение): Получить данные из выбранного источника (CSV, JSON, API, база данных и т.д.).

Transform (Трансформация): Очистить, отфильтровать и агрегировать данные.

Load (Загрузка): Сохранить обработанные данные в хранилище (БД, облако, файл).

Автоматизировать процесс с использованием инструментов (Python, Apache Airflow и др.).

Инструменты и технологии

Язык программирования: Python (библиотеки: Pandas, SQLAlchemy, Requests).

Инструменты ETL: Apache Airflow (для оркестрации), Jupyter Notebook (для тестирования).

Хранилища данных: PostgreSQL, SQLite, Google BigQuery или CSV/Parquet-файлы.

Дополнительно: Docker (для контейнеризации), Git (для контроля версий).

Шаги выполнения

1. Выбор источника данных

Примеры данных:

Датчики IoT (температура, влажность).

Логи веб-приложений.

Открытые datasets (Kaggle, Google Dataset Search).

2. Извлечение данных (Extract)

Написать скрипт на Python для чтения данных:

```
python
```

```
import pandas as pd
```

Пример для CSV

```
data = pd.read_csv('source_data.csv')
```

Пример для API

```
import requests
```

```
response = requests.get('https://api.example.com/data')
```

```
data = response.json()
```

3. Трансформация данных (Transform)

Обработка данных:

Удаление дубликатов.

Заполнение пропущенных значений.

Фильтрация аномалий.

Агрегация (например, вычисление средних значений за час).

python

Пример очистки данных

```
data_clean = data.drop_duplicates().dropna()
```

Пример агрегации

```
data_transformed = data_clean.groupby('sensor_id').mean()
```

4. Загрузка данных (Load)

Сохранение в целевую систему:

python

В CSV

```
data_transformed.to_csv('processed_data.csv')
```

В базу данных PostgreSQL

```
from sqlalchemy import create_engine
```

```
engine = create_engine('postgresql://user:password@localhost:5432/mydb')
```

```
data_transformed.to_sql('sensor_data', engine, if_exists='replace')
```

5. Автоматизация пайплайна

Использование Apache Airflow для создания расписания и зависимостей задач:

```
python

from airflow import DAG

from airflow.operators.python_operator import PythonOperator

from datetime import datetime


def etl_task():

    # Код ETL


dag = DAG('etl_pipeline', schedule_interval='@daily', start_date=datetime(2023, 1, 1))

task = PythonOperator(task_id='run_etl', python_callable=etl_task, dag=dag)
```

Пример результата

Репозиторий на GitHub с кодом ETL-пайплайна.

Дашборд в Tableau/Power BI на основе обработанных данных.

Логи выполнения и отчет об ошибках (если есть).

Советы

Реализуйте обработку ошибок (try/except) и логирование.

Протестируйте пайплайн на небольших данных перед масштабированием.

Используйте Docker для изоляции окружения (например, Airflow + PostgreSQL).

Вопросы для защиты

Какие этапы ETL являются самыми ресурсозатратными и почему?

Как обеспечить идемпотентность пайплайна (повторяемость без дублирования)?

Какие метрики качества данных вы контролировали?

Готовый пайплайн можно использовать для анализа данных в реальном времени, прогнозирования или бизнес-отчетности.